Dr. D. Y. Patil Educational Federation's
Dr. D. Y. PATIL COLLEGE OF ENGINEERING & INNOVATION
Survey No. 27/A/1/2C, Varale Campus, Near Talegaon Railway Station,
Tal. Maval, Dist. Pune 410 507, Ph: 020 48522561, 565,566
Web Site: www.dypcoei.edu.in

Subject: Laboratory Practice VI (410256)

Natural Language Processing Laboratory

Course: - 2019

Assignment No. 02

Name of the Student:_____Roll no:_____

CLASS: - B.E. [Computer] **Division: A/B/C**

Marks: _____/10

Date of Checking: ___/____/_____ **Sign with Date: _____**

ASSIGNMENT NO: 02

TITLE:
To study bag-of-words approach on text data using python.

AIM:
Perform bag-of-words approach (count occurrence, normalized count occurrence), TF-IDF on data. Create embeddings using Word2Vec.

Theory:

1. Bag-of-Words Approach

In the Bag-of-Words (BoW) approach, the text is represented as a collection of words where the order does not matter, and each word's frequency is counted.

Steps:

1. **Tokenization**: Split the text into individual words.
2. **Count Occurrence**: Count how many times each word appears in the document.
3. **Normalized Count Occurrence**: Divide the count of each word by the total number of words in the document, essentially normalizing the frequency.

In Python, you can use libraries like CountVectorizer from sklearn to achieve this.

Python code:

```
from sklearn.feature_extraction.text import CountVectorizer
import numpy as np
# Sample corpus
corpus = [
   "The Cat sat on the mat",
   "The dog sat on the rug"
]
# Step 1: Count Vectorizer
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
# Convert to array and normalize
word_counts = X.toarray()
normalized_counts = word_counts / np.sum(word_counts, axis=1,
keepdims=True)
print("Word counts:\n", word_counts)
print("Normalized word counts:\n", normalized_counts)
```

Explaination:
Vocabulary Creation:
['cat', 'dog', 'mat', 'on', 'rug', 'sat', 'the']

Word count:
Sentence – 1 [[1 0 1 1 0 1 2]
Sentence – 2 [0 1 0 1 1 1 2]]

Total words in each sentence:
1 + 0 + 1 + 1 + 0 + 1 + 2 = 7
0 + 1 + 0 + 1 + 1 + 1 + 2 = 7

TF calculation:
TF = frequency of word in sentence / total words in sentence

Word	S1	S2
cat	1/7	0
dog	0	1/7
mat	1/7	0
on	1/7	1/7
rug	0	1/7
sat	1/7	1/7
the	2/7	2/7

[[0.14 0.00 0.14 0.14 0.00 0.14 0.28]
[0.00 0.14 0.00 0.14 0.14 0.14 0.28]]

2. TF-IDF (Term Frequency-Inverse Document Frequency)
TF-IDF measures the importance of a word in a document relative to a corpus. It is calculated by multiplying Term Frequency (TF) by Inverse Document Frequency (IDF).
Steps:

1. **TF**: Term Frequency is the number of times a word appears in a document.
2. **IDF**: Inverse Document Frequency gives more weight to words that appear less frequently across the corpus.

You can use TfidfVectorizer from sklearn to calculate TF-IDF.

Python code:

```
from sklearn.feature_extraction.text import TfidfVectorizer
# Step 2: TF-IDF Vectorizer
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(corpus)
print("TF-IDF Matrix:\n", tfidf_matrix.toarray())
```

Inverse Document Frequency (IDF)

Formula:
IDF = log (N / df)

Word	df	IDF
cat	**1**	**log(2/1) ≈ 0.693**
dog	**1**	**0.693**
mat	**1**	**0.693**
rug	**1**	**0.693**
on	**2**	**log(2/2) = 0**
sat	**2**	**0**
the	**2**	**0**

TF-IDF = TF × IDF

Sentence 1:
cat → (1/7)*0.693 ≈ 0.099
mat → (1/7)*0.693 ≈ 0.099
others → 0

Sentence 2:
dog → (1/7)*0.693 ≈ 0.099
rug → (1/7)*0.693 ≈ 0.099
others → 0

TF-IDF Matrix:

```
[[0.09 0.00  0.09  0.00  0.00  0.00  0.00]
 [0.00  0.09  0.00  0.00  0.09  0.00  0.00]]
```

Normalize using:

$$\sqrt{(0.099)^2 + (0.099)^2} = 0.14$$

0.099 / 0.14 ≈ 0.577

So, Normalized TF-IDF Matrix:

```
[[0.57  0.00  0.57  0.00  0.00  0.00  0.00]
 [0.00  0.57  0.00  0.00  0.57  0.00  0.00]]
```

3. Word2Vec Embeddings

Word2Vec creates vector representations of words based on their context. It can be trained on a corpus and then used to represent words in a dense vector space.

Steps:

1. **Tokenization**: Split text into tokens (words).
2. **Train Word2Vec Model**: Train a Word2Vec model using a corpus of text.
3. **Obtain Word Embeddings**: Get the vector representation of a word.

You can use gensim's Word2Vec class to train embeddings.

Python code:

```
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize
# Sample corpus
corpus = [
   "I love machine learning",
   "Machine learning is fascinating",
   "I love programming"
]
# Step 3: Tokenize sentences
tokenized_corpus = [word_tokenize(sentence.lower()) for sentence in
corpus]
# Train Word2Vec model
model = Word2Vec(tokenized_corpus, vector_size=100, window=5,
min_count=1, sg=0)
# Example: Get word embedding for 'machine'
word_embedding = model.wv['machine']
print("Word embedding for 'machine':", word_embedding)
```

Conclusion: We have study bag-of-words approach on text data.

1. **Bag-of-Words**: Count word frequencies and normalize them.
2. **TF-IDF**: Calculate term importance across a corpus.
3. **Word2Vec**: Generate dense word embeddings from context.

Questions:

Q1. Expalin Tokenization in details.
Q2. Explain Stages of NLP in details.
Q3. Expalin Part of Speech Tagging in details.